

TP Semaine 7 — Correction intégrale

1. Objectif du TP

L'objectif est de construire un modèle de **scoring de crédit** capable de prédire si un client va **rembourser** son crédit (classe 0) ou **faire défaut** (classe 1). Le travail suit un pipeline classique de machine learning : exploration des données, création de variables, prétraitement, entraînement, validation croisée et sauvegarde du modèle final.

2. Partie 1 — Exploration des données

Cette première partie consiste à vérifier la structure du dataset, repérer les valeurs manquantes et observer la répartition de la cible.

Cellule 1.2 — Exploration rapide

```
# 1. Affichez le résumé des types et valeurs manquantes print("=== Info dataset ===")
df.info()

# 2. Comptez les valeurs manquantes par colonne print("\n=== Valeurs manquantes ===")
print(df.isna().sum())

# 3. Distribution de la cible
print("\n=== Distribution de la cible ===")
print(df['default'].value_counts())
```

```
print(df['default'].value_counts(normalize=True).round(3))
```

Résultats

attendus :

- revenu_mensuel contient 15 valeurs manquantes •
score_bureau contient 30 valeurs manquantes •
La cible default contient 780 remboursements et 220 défauts
- Le taux de défaut est de 22 %

Réponse rédigée : Les colonnes contenant des valeurs manquantes sont revenu_mensuel et score_bureau. Le dataset n'est pas parfaitement équilibré : il contient 78 % de remboursements et 22 % de défauts.

Cellule 1.3 — Taux de défaut par statut d'emploi

```
fig, ax = plt.subplots(figsize=(8, 4))

# Calculez le taux de défaut par statut_emploi
taux_defaut = df.groupby('statut_emploi')['default'].mean().sort_values

# Tracez le barplot
sns.barplot(x=taux_defaut.index, y=taux_defaut.values, ax=ax)
ax.set_title('Taux de défaut par statut d'emploi')
ax.set_ylabel('Taux de défaut')
ax.set_xlabel('Statut d'emploi')
plt.tight_layout()
plt.show()
```

Interprétation : Le statut CDD présente ici le taux de défaut observé le plus élevé, devant CDI, Sans emploi et

Indépendant sur ce dataset simulé.

Réponse rédigée : Le statut d'emploi ayant le taux de défaut le plus élevé est CDD.

3. Partie 2 — Feature Engineering

Dans cette partie, on enrichit le dataset avec de nouvelles variables susceptibles d'améliorer la qualité prédictive du modèle.

Cellule 2.1 — Création de trois nouvelles features

```
df_feat = df.copy()

# Feature 1 : taux_endettement = montant_credit / (revenu_mensuel *
dudf_feat['taux_endettement'] = df_feat['montant_credit'] / (df_feat['re

# Feature 2 : revenu_log = log1p(revenu_mensuel)
df_feat['revenu_log'] = np.log1p(df_feat['revenu_mensuel'])

# Feature 3 : profil_risque = 1 si nb_incidents_prec > 0 ET score_bure# sinon 0
df_feat['profil_risque'] = ((df_feat['nb_incidents_prec'] > 0) & (df_f

# Vérification : affichez les 5 premières lignes des nouvelles
featureprint(df_feat[['taux_endettement', 'revenu_log', 'profil_risque']].hea

# Corrélation des nouvelles features avec la cible
print("\nCorrélation avec 'default' :")
for feat in ['taux_endettement', 'revenu_log', 'profil_risque']:
    r = df_feat[feat].corr(df_feat['default'])
    print(f' {feat}:<20} : r = {r:+.4f}')
```

Feature	Corrélation avec default
taux_endettement	+0.0563
revenu_log	-0.1426

profil_risque	+0.2173
---------------	---------

Réponse rédigée : La feature la plus corrélée avec le défaut est profil_risque, ce qui confirme qu'une combinaison d'incidents passés et d'un score bureau faible constitue un signal de risque pertinent.

Cellule 2.2 — Discrétisation de l'âge

```
df_feat['tranche_age'] = pd.cut(
    df_feat['age'],
    bins = [20, 30, 45, 55, np.inf],
    labels = ['Jeune (20-30)', 'Adulte (30-45)', 'Senior (45-55)', 'Re right = False
    )

# Vérification : taux de défaut par tranche
print("Taux de défaut par tranche d'âge :")
print(df_feat.groupby('tranche_age', observed=True)['defaut'].mean()).I
```

Lecture des résultats : Les taux de défaut par tranche d'âge restent proches les uns des autres, ce qui suggère que l'âge seul n'est pas ici un facteur discriminant majeur.

Cellule 2.3 — Vérification du skewness

```
from scipy.stats import skew

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# Histogramme du revenu original
sns.histplot(df_feat['revenu_mensuel'].dropna(), bins=30, kde=True,
axaxes[0].set_title(f'Revenu original\nskewness = {skew(df_feat["revenu_

# Histogramme du revenu transformé
sns.histplot(df_feat['revenu_log'].dropna(), bins=30, kde=True,
ax=axeaxes[1].set_title(f'log1p(Revenu)\nskewness = {skew(df_feat["revenu_lo

plt.suptitle('Effet de la transformation logarithmique', fontweight='bplt.tight_layout()
```

```
plt.show()
```

Interprétation : Le skewness passe d'environ **1.91** à **0.00** après transformation logarithmique. Cela rend la distribution du revenu plus symétrique et plus facile à exploiter pour certains modèles.

4. Partie 3 — Construction du pipeline

Cette étape met en place le prétraitement automatique des données numériques et catégorielles avant l'entraînement des modèles.

Cellule 3.1 — Séparation des variables et split

```
# Features sélectionnées (incluant les nouvelles de la Partie 2) FEATURES_NUM =
['revenu_log', 'montant_credit', 'duree_mois', 'nb_credits_prec', 'nb_incidents_prec',
'score_bureau', 'taux_endettement', 'profil_risque']

FEATURES_CAT = ['statut_emploi', 'region', 'a_compte_epargne'] ALL_FEATURES
= FEATURES_NUM + FEATURES_CAT

TARGET = 'default'

X = df_feat[ALL_FEATURES]
y = df_feat[TARGET]

# Split 80/20, stratifié sur y
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    stratify=y,
    random_state=42
)

print(f"Train : {X_train.shape[0]} clients ({y_train.mean()*100:.1f}% print(f"Test :
{X_test.shape[0]} clients ({y_test.mean()*100:.1f}% d
```

Le split stratifié conserve la même proportion de défauts dans l'échantillon d'entraînement et dans l'échantillon de test, ce qui garantit une évaluation plus fiable.

Cellule 3.2 — ColumnTransformer

```
# Transformateur pour les features numériques :
# → imputer les NaN par la médiane, puis StandardScaler
num_transformer = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler()),
])

# Transformateur pour les features catégorielles :
# → imputer les NaN par le mode, puis OneHotEncoder
cat_transformer = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')), ('ohe', OneHotEncoder(drop='first',
sparse_output=False, handle))

# Assemblage
preprocessor = ColumnTransformer(transformers=[
    ('num', num_transformer, FEATURES_NUM),
    ('cat', cat_transformer, FEATURES_CAT),
])
# Test rapide
X_sample = preprocessor.fit_transform(X_train.head(10)) print(f"Shape avant
preprocessing : {X_train.head(10).shape}") print(f"Shape après preprocessing :
{X_sample.shape}") print("NaN restants :", np.isnan(X_sample).sum())
```

Réponse rédigée : Le nombre de colonnes augmente après le preprocessing parce que les variables catégorielles sont transformées en plusieurs colonnes binaires via le *One-Hot Encoding*.

Cellule 3.3 — Deux pipelines à comparer

```

# Pipeline 1 — Régression Logistique
pipe_lr = Pipeline([
    ('preprocessor', preprocessor),
    ('model', LogisticRegression(class_weight='balanced', max_iter=100))

# Pipeline 2 — Arbre de Décision
pipe_tree = Pipeline([
    ('preprocessor', preprocessor),
    ('model', DecisionTreeClassifier(max_depth=5, class_weight='balanc]

# Entraînement
pipe_lr.fit(X_train, y_train)
pipe_tree.fit(X_train, y_train)

# Évaluation sur le test set
for nom, pipe in [('Régression Logistique', pipe_lr), ('Arbre de Décis y_pred =
pipe.predict(X_test)
y_proba = pipe.predict_proba(X_test)[:, 1]
auc = roc_auc_score(y_test, y_proba)

print(f"\n{'—'*45}")
print(f" {nom}")
print(f" AUC-ROC : {auc:.4f}")

print(classification_report(y_test, y_pred, target_names=['Rembour

```

Modèle	AUC test	Recall défaut	F1 défaut	Accuracy
Régression Logistique	0.6898	0.5455	0.4068	0.65
Arbre de Décision	0.6246	0.5682	0.4167	0.65

Réponse rédigée : L'arbre de décision obtient le meilleur Recall sur la classe Défaut. Cette métrique est essentielle pour la MFC, car elle mesure la capacité à détecter les clients réellement risqués et limite le coût lié aux faux négatifs.

5. Partie 4 — Validation croisée

La validation croisée permet de comparer les modèles de manière plus robuste et de détecter un éventuel surapprentissage.

Cellule 4.1 — Comparaison avec `cross_validate`

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42) resultats_cv = {}

for nom, pipe in [('Log. Regression', pipe_lr), ('Decision Tree', pipe_dt)]:
    cv_res = cross_validate(
        pipe,
        X, y,
        cv=cv,
        scoring=['f1', 'recall', 'roc_auc'],
        return_train_score=True,
        n_jobs=-1,
    )
    resultats_cv[nom] = cv_res

# Affichage des résultats
print(f"{'Modèle':<22} {'Recall (CV)':>14} {'F1 (CV)':>12} {'AUC (CV)':>12}")

for nom, res in resultats_cv.items():
    recall_test = res['test_recall'].mean()
    recall_train = res['train_recall'].mean()
    f1_test = res['test_f1'].mean()
    auc_test = res['test_roc_auc'].mean()
    gap = recall_train - recall_test
    print(f"{'nom':<22} {'recall_test':>12.4f} {'f1_test':>12.4f} {'auc_test':>12.4f} {'gap':>12.4f}")
```

Modèle	Recall CV	F1 CV	AUC CV	Gap Recall
Log. Regression	0.5818	0.4357	0.6858	+0.0284
Decision Tree	0.6364	0.4582	0.6922	+0.1034

Réponses rédigées : Le gap train/test de l'arbre de décision est plus élevé, ce qui peut signaler un surapprentissage modéré. Malgré cela, ce modèle reste intéressant pour la MFC car il obtient le meilleur rappel moyen en validation croisée, ce qui correspond bien à l'objectif métier.

Cellule 4.2 — Courbe de validation sur la profondeur

```
depths = [1, 2, 3, 4, 5, 6, 7, 8, 10, 12]

train_scores, val_scores = validation_curve(
    pipe_tree,
    X, y,
    param_name = 'model__max_depth',
    param_range = depths,
    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=0),
    scoring = 'recall',
    n_jobs = -1,
)

train_m = train_scores.mean(axis=1)
val_m = val_scores.mean(axis=1)
best_d = depths[np.argmax(val_m)]

# Tracé
fig, ax = plt.subplots(figsize=(9, 4))
ax.plot(depths, train_m, 'b-o', lw=2, ms=6, label='Recall Train')
ax.plot(depths, val_m, 'r-o', lw=2, ms=6, label='Recall Validation')
ax.axvline(best_d, color='green', ls='--', lw=2,
           label=f'Profondeur optimale = {best_d}')
ax.set_xlabel('max_depth')
ax.set_ylabel('Recall')
ax.set_title('Courbe de validation — Arbre de Décision (MFC)', fontweight='bold')
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print(f"Profondeur optimale : {best_d}")
print(f"Recall validation max : {max(val_m):.4f}")
```

Réponse rédigée : La profondeur optimale trouvée est 5. Elle

est donc identique à celle choisie initialement dans le pipeline.

6. Partie 5 — Sauvegarde du modèle

La dernière étape consiste à réentraîner le meilleur modèle et à sauvegarder à la fois le pipeline et un petit rapport JSON.

```
import pickle, json, os
from datetime import datetime

os.makedirs('resultats', exist_ok=True)

# Ré-entraîner l'arbre avec la profondeur optimale trouvée pipe_final =
Pipeline([
    ('preprocessor', preprocessor),
    ('model', DecisionTreeClassifier(
        max_depth = best_d,
        class_weight = 'balanced',
        random_state = 42
    )),
])
pipe_final.fit(X_train, y_train)

# Sauvegarde
with open('resultats/modele_mfc.pkl', 'wb') as f:
    pickle.dump(pipe_final, f)

# Métriques finales sur le test set
y_pred_final = pipe_final.predict(X_test)
y_proba_final = pipe_final.predict_proba(X_test)[:, 1]

rapport = {
    'modele' : f'DecisionTree depth={best_d}',
    'date' : datetime.now().strftime('%Y-%m-%d %H:%M'), 'auc_test' :
    round(roc_auc_score(y_test, y_proba_final), 4), 'features' : ALL_FEATURES,
    'n_train' : int(X_train.shape[0]),
}
with open('resultats/rapport_mfc.json', 'w') as f:
    json.dump(rapport, f, indent=2)
```

```
print("✓ Modèle sauvegardé : resultats/modele_mfc.pkl") print(f"✓ AUC  
finale : {rapport['auc_test']}")  
print(f"✓ Profondeur : {best_d}")
```

Résultat final : le modèle retenu est un
DecisionTreeClassifier de profondeur 5, avec une AUC
test finale d'environ **0.6246**.

7. Réponses synthétiques à copier

Valeurs manquantes et équilibre : Les colonnes contenant des valeurs manquantes sont revenu_mensuel et score_bureau. Le dataset est modérément déséquilibré avec 22 % de défauts.

Statut le plus risqué : Le statut d'emploi CDD présente le taux de défaut le plus élevé.

Feature la plus corrélée : La variable profil_risque est la plus corrélée avec le défaut.

Augmentation du nombre de colonnes : Elle s'explique par le *One-Hot Encoding* appliqué aux variables catégorielles.

Meilleur recall : L'arbre de décision obtient le meilleur recall pour la classe Défaut, ce qui est essentiel pour détecter un maximum de clients risqués.

Gap train/test : Le gap de l'arbre de décision est plus élevé et suggère un surapprentissage modéré.

Modèle recommandé : Un arbre de décision de profondeur 5 constitue un bon compromis entre performance et simplicité.

Profondeur optimale : La profondeur optimale trouvée par

validation est 5, identique à celle fixée au départ.

8. Conclusion

Ce TP a permis de construire un pipeline complet de scoring de crédit pour la MFC. Après une phase d'exploration, plusieurs variables pertinentes ont été créées afin d'améliorer l'information disponible pour le modèle. Deux approches ont été comparées : la régression logistique et l'arbre de décision. Même si la régression logistique est plus stable, l'arbre de décision de profondeur 5 offre le meilleur rappel sur la classe des défauts, ce qui répond mieux au besoin métier de détection du risque de non-remboursement. Le modèle final a ensuite été sauvegardé avec ses principales métadonnées pour une utilisation ultérieure.

Document préparé en français, formaté pour un rendu propre, lisible et facilement exportable en PDF.